

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 10%

QUESTIONS: 10

**Duration :60 Mins
In Class
Total Marks:50**

Annexure A

DATA INTERPRETATION

Code of Conduct

*I _____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Annexure A

DATA INTERPRETATION

Q1. Given the three-address code:

```
t1 = a * b
t2 = t1 + c
t3 = t2 - d
```

- (i) Analyze the sequence of computations performed by the intermediate code.
- (ii) Construct the equivalent expression tree for the given code.
- (iii) Explain how temporary variables (t1, t2) help reduce recomputation and improve execution efficiency. (5 Marks)

Q2. Given the symbol table entries:

Variable	Type	Size
p	int	4
q	float	8
r	char	1
s	double	8

- (i) Interpret how memory is allocated for each variable.
- (ii) Calculate the total memory requirement for the variables.
- (iii) Explain the effect of memory alignment and padding on the overall storage size. (5 Marks)

Q3. Given the intermediate code:

```
t1 = x + y
t2 = t1 * z
t3 = x + y
t4 = t3 - w
```

- (i) Trace the computation step-by-step.
- (ii) Identify opportunities for common subexpression elimination.
- (iii) Rewrite the optimized intermediate code after removing redundant computations. (5 Marks)

Q4. Given the condition:

if (a < b || c > d)

Intermediate code:

```
if a < b goto L1
if c > d goto L1
goto L2
L1: ...
```

- (i) Interpret the control flow represented by the intermediate code.
- (ii) Explain how short-circuit evaluation is implemented.
- (iii) Illustrate the execution flow for different values of a, b, c, and d. (5 Marks)

Q5. Given:

Instruction	Code
120	if p > q goto _
121	goto _

Backpatch lists:

```
truelist = {120}
falselist = {121}
```

- (i) Analyze how backpatching is applied to the given instructions.
- (ii) Fill in the appropriate target labels for the pending jumps.
- (iii) Explain how control flow is resolved using the truelist and falselist. (5 Marks)

Q6. Given the expression:

result = a * b + c / d

- (i) Generate the corresponding three-address code.
- (ii) Explain the order of evaluation based on operator precedence.
- (iii) Describe the role of temporary variables in evaluating the expression. (5 Marks)

Q7. Given:

```
switch(choice)
{
    case 1: OPEN;
```

```
case 2: SAVE;  
case 3: EXIT;  
}
```

- (i) Translate the switch statement into intermediate code using conditional jumps.
- (ii) Explain the execution of conditional branches for each case.
- (iii) Describe how a jump table can optimize the execution of the switch statement.

(5 Marks)

Q8. Given the function call:

total = compute(x, y)

Intermediate representation:

```
param x  
param y  
call compute, 2
```

- (i) Explain the parameter passing mechanism used in the procedure call.
- (ii) Describe the structure and purpose of the activation record.
- (iii) Explain how the return value is transferred back to the calling function. (5 Marks)

Q9. Given:

if (m > n && p < q)

Intermediate code:

```
if m > n goto L1  
goto L3
```

```
L1: if p < q goto L2  
goto L3
```

- (i) Interpret the control flow represented by the code.
- (ii) Explain how true and false lists are maintained during code generation.
- (iii) Describe how backpatching resolves unresolved jump targets. (5 Marks)

Q10. Given the intermediate code:

```
t1 = p * q  
t2 = t1 + r  
t3 = t2 * s
```

- (i) Analyze the data dependencies among the statements.
- (ii) Explain how computed values are reused through temporary variables.
- (iii) Construct the corresponding data-flow representation and identify possible optimization opportunities. (5 Marks)

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding ; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q1. Given the three-address code:

$t1 = a * b$

$t2 = t1 + c$

$t3 = t2 - d$

(i) Analyze the sequence of computations performed by the intermediate code.

Answer:

The intermediate code executes the computations in the following sequence:

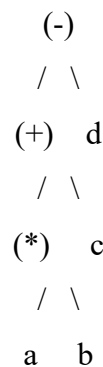
1. Multiply **a** and **b** and store the result in **t1**.
2. Add **c** to **t1** and store the result in **t2**.
3. Subtract **d** from **t2** and store the final result in **t3**.

The equivalent mathematical expression is:

$((a \times b) + c) - d$

(ii) Construct the equivalent expression tree for the given code.

Answer:



(iii) Explain how temporary variables (t1, t2) help reduce recomputation and improve execution efficiency.

Answer:

- Temporary variables store intermediate results.
- They avoid repeated calculations of the same expression.
- They reduce CPU execution time.
- They simplify intermediate code generation.
- They improve compiler optimization and overall execution efficiency.

Q2. Given the symbol table entries:

Variable	Type	Size
p	int	4 Bytes
q	float	8 Bytes

	Variable	Type	Size
r		char	1 Byte
s		double	8 Bytes

(i) Interpret how memory is allocated for each variable.

Answer:

Memory is allocated according to the data type of each variable.

- **p (int)** occupies **4 bytes**.
- **q (float)** occupies **8 bytes**.
- **r (char)** occupies **1 byte**.
- **s (double)** occupies **8 bytes**.

Each variable is assigned memory space based on its size in the symbol table.

(ii) Calculate the total memory requirement for the variables.

Answer:

Total Memory Required:

= Size of p + Size of q + Size of r + Size of s

= 4 + 8 + 1 + 8

= 21 Bytes

Therefore, the total memory required is **21 bytes**.

(iii) Explain the effect of memory alignment and padding on the overall storage size.

Answer:

Memory alignment allows the processor to access data efficiently by storing variables at properly aligned memory addresses. Since **char** occupies only one byte, the compiler may insert **padding bytes** before the **double** variable to maintain proper alignment. As a result, the actual storage size may be greater than **21 bytes**, depending on the compiler and system architecture.

Q3. Given the intermediate code:

t1 = x + y

t2 = t1 * z

t3 = x + y

$t4 = t3 - w$

(i) Trace the computation step-by-step.

Answer:

The computations are performed as follows:

1. Compute $x + y$ and store the result in **t1**.
2. Multiply **t1** with z and store the result in **t2**.
3. Compute $x + y$ again and store the result in **t3**.
4. Subtract w from **t3** and store the result in **t4**.

(ii) Identify opportunities for common subexpression elimination.

Answer:

The expression

$x + y$

is calculated twice.

Since the expression produces the same value and the operands do not change, it is a **common subexpression**. The second computation is unnecessary and can be eliminated.

(iii) Rewrite the optimized intermediate code after removing redundant computations.

Answer:

$t1 = x + y$

$t2 = t1 * z$

$t4 = t1 - w$

Advantages:

- Eliminates redundant computations.
- Reduces execution time.
- Produces efficient intermediate code.
- Improves compiler optimization.

Q4. Given the condition:

if ($a < b \parallel c > d$)

Intermediate Code:

if $a < b$ goto L1

if $c > d$ goto L1

goto L2

L1:

...

(i) Interpret the control flow represented by the intermediate code.

Answer:

The control flow executes as follows:

1. The condition $a < b$ is evaluated first.
2. If it is true, execution jumps directly to **L1**.
3. If it is false, the condition $c > d$ is evaluated.
4. If $c > d$ is true, execution jumps to **L1**.
5. If both conditions are false, execution jumps to **L2**.

Thus, the program reaches **L1** if either condition is true.

(ii) Explain how short-circuit evaluation is implemented.

Answer:

Short-circuit evaluation stops evaluating expressions as soon as the final result is known.

For the logical OR operator ($\parallel$):

- If $a < b$ is true, the second condition $c > d$ is not evaluated.
- This reduces unnecessary computations and improves execution speed.

(iii) Illustrate the execution flow for different values of a, b, c, and d.

Answer:

Case 1

$a = 2$

$b = 5$

$c = 10$

$d = 20$

Evaluation:

$2 < 5 \rightarrow \text{True}$

Execution jumps directly to **L1**.

Case 2

$a = 8$

$b = 5$

c = 12

d = 4

Evaluation:

$8 < 5 \rightarrow \text{False}$

$12 > 4 \rightarrow \text{True}$

Execution jumps to **L1**.

Case 3

a = 8

b = 5

c = 2

d = 9

Evaluation:

$8 < 5 \rightarrow \text{False}$

$2 > 9 \rightarrow \text{False}$

Execution jumps to **L2**.

Q5. Given:

Instruction Code

120 if p > q goto _

121 goto _

Backpatch Lists

truelist = {120}

falselist = {121}

(i) Analyze how backpatching is applied to the given instructions.

Answer:

Initially, the compiler does not know the destination labels of the jump instructions. Therefore, the target fields are left blank (_). The instruction numbers are stored in the **truelist** and **falselist**. After the labels are generated, the compiler fills the correct addresses into these incomplete jump instructions. This process is called **backpatching**.

(ii) Fill in the appropriate target labels for the pending jumps.

Answer:

One possible completed code is:

120 if $p > q$ goto L1

121 goto L2

Where:

- **L1** is the target when the condition is true.
- **L2** is the target when the condition is false.

(iii) Explain how control flow is resolved using the truelist and falselist.

Answer:

The truelist contains jump instructions that should transfer control when the condition evaluates to true. The falselist contains jump instructions that should transfer control when the condition evaluates to false.

During backpatching:

- Every instruction in the **truelist** is updated with the address of the true label (**L1**).
- Every instruction in the **falselist** is updated with the address of the false label (**L2**).

This resolves all pending jumps and establishes the correct control flow of the program.

Q6. Given the expression:

$\text{result} = a * b + c / d$

(i) Generate the corresponding three-address code.

Answer:

The three-address code for the given expression is:

$t1 = a * b$

$t2 = c / d$

$t3 = t1 + t2$

$\text{result} = t3$

Each instruction performs one operation and stores the result in a temporary variable.

(ii) Explain the order of evaluation based on operator precedence.

Answer:

According to operator precedence:

1. Multiplication (*) is performed first.
2. Division (/) is performed next.
3. Addition (+) is performed last.

Therefore, the expression is evaluated as:

$$(a \times b) + (c \div d)$$

This ensures the correct result according to arithmetic precedence rules.

(iii) Describe the role of temporary variables in evaluating the expression.

Answer:

Temporary variables store intermediate results during computation.

- **t1** stores the result of **a × b**.
- **t2** stores the result of **c ÷ d**.
- **t3** stores the sum of **t1** and **t2**.

Using temporary variables avoids repeated computations, simplifies code generation, and improves execution efficiency.

Q7. Given:

```
switch(choice)
{
case 1: OPEN;
case 2: SAVE;
case 3: EXIT;
}
```

(i) Translate the switch statement into intermediate code using conditional jumps.

Answer:

```
if choice == 1 goto L1
if choice == 2 goto L2
if choice == 3 goto L3
goto L4
```

L1:

OPEN

goto END

L2:

SAVE

goto END

L3:

EXIT

goto END

L4:

END:

The compiler compares the value of **choice** with each case and transfers control to the matching label.

(ii) Explain the execution of conditional branches for each case.

Answer:

- If **choice** = 1, control transfers to **L1**, and **OPEN** is executed.
- If **choice** = 2, control transfers to **L2**, and **SAVE** is executed.
- If **choice** = 3, control transfers to **L3**, and **EXIT** is executed.
- If none of the conditions are satisfied, execution proceeds to **L4** (default/end).

After executing a case, control jumps to **END** to avoid executing the remaining cases.

(iii) Describe how a jump table can optimize the execution of the switch statement.

Answer:

A jump table stores the addresses of all case labels.

Instead of comparing every case one by one, the compiler directly jumps to the appropriate instruction using the value of **choice**.

Advantages:

- Faster execution.
- Constant-time branching.
- Suitable for switch statements with many cases.
- Reduces the number of comparisons.

Q8. Given the function call:

total = compute(x, y)

Intermediate Representation:

param x

param y

call compute, 2

(i) Explain the parameter passing mechanism used in the procedure call.

Answer:

The parameters **x** and **y** are passed to the function before the function call.

param x

param y

call compute, 2

Here,

- param x passes the first argument.
- param y passes the second argument.
- call compute,2 calls the function with two parameters.

(ii) Describe the structure and purpose of the activation record.**Answer:**

An activation record (stack frame) stores information required during function execution.

It contains:

- Function parameters.
- Local variables.
- Return address.
- Temporary variables.
- Saved registers.

The activation record helps the function execute correctly and allows control to return to the calling function after execution.

(iii) Explain how the return value is transferred back to the calling function.**Answer:**

After completing execution, the function returns the computed value to the calling function.

Example:

total = return_value

The returned value is stored in the variable **total**, allowing the calling program to use the result.

Q9. Given:

if (m > n && p < q)

Intermediate Code:

if m > n goto L1

```
goto L3
L1:
if p < q goto L2
goto L3
```

(i) Interpret the control flow represented by the code.

Answer:

The control flow is executed as follows:

1. Evaluate $m > n$.
2. If false, jump directly to **L3**.
3. If true, evaluate $p < q$.
4. If $p < q$ is true, jump to **L2**.
5. Otherwise, jump to **L3**.

Thus, both conditions must be true for execution to reach **L2**.

(ii) Explain how true and false lists are maintained during code generation.

Answer:

During code generation:

- The **True List** stores jump instructions that should execute when the condition is true.
- The **False List** stores jump instructions that should execute when the condition is false.

These lists contain incomplete jump addresses that are updated later by the compiler.

(iii) Describe how backpatching resolves unresolved jump targets.

Answer:

Initially, jump instructions contain blank target addresses.

Backpatching replaces these blank addresses with the correct labels once they become available.

For example:

Before backpatching:

```
goto _
```

After backpatching:

```
goto L2
```

or

```
goto L3
```

This completes the control-flow graph and ensures correct program execution.

Q10. Given the intermediate code:

$t1 = p * q$

$t2 = t1 + r$

$t3 = t2 * s$

(i) Analyze the data dependencies among the statements.

Answer:

The dependencies are:

- **t1** depends on **p** and **q**.
- **t2** depends on **t1** and **r**.
- **t3** depends on **t2** and **s**.

Each statement uses the result produced by the previous statement.

(ii) Explain how computed values are reused through temporary variables.

Answer:

Temporary variables store intermediate results so that they can be reused without recalculating expressions.

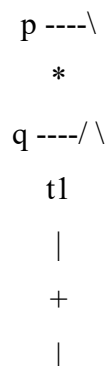
- **t1** stores the multiplication result.
- **t2** stores the addition result.
- **t3** uses the previously computed value.

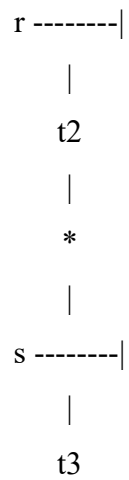
This minimizes redundant computations and improves execution efficiency.

(iii) Construct the corresponding data-flow representation and identify possible optimization opportunities.

Answer:

Data-Flow Representation:





Optimization Opportunities:

- Reuse temporary variables effectively.
- Eliminate redundant computations.
- Improve register allocation.
- Apply constant folding when operands are constants.
- Reduce memory accesses through efficient data flow.